Relatório no âmbito da Meta 2 do projeto de Sistemas Distribuídos

João André Pinheiro Canais
2023213000
uc2023213000@student.uc.pt

Francisco Carvalho Gouveia
2023214517
uc2023214517@student.uc.pt

1. Testes de software 3
2. Arquitetura de Software: Integração FastAPI com Servidor RPC/RMI 5
   Padrão MVC 5
   Camada de Serviços: A Ponte entre HTTP e RPC 6
3. Integração de WebSockets 6
4. Integração de REST Web Services 7
   4.1. Integração com Hacker News API 7
   4.2. Integração com IA Generativa (Ollama/Gemini/Groq) 7
5. Distribuição de tarefas pelos elementos do grupo 8

1. Testes de software

Caso de teste	Descrição	Resultado esperado	Estado
Indexação Manual de URL	Aceder a /index-link, inserir um URL válido (ex: https://example.com) e submeter.	O sistema deve informar que o URL foi enviado para a fila. O *Downloader* deve processar o URL e o *Barrel* deve persistir os dados.	Pass
Pesquisa de Palavras	Na *Homepage*, pesquisar por um termo conhecido (ex: "python").	Devem ser apresentados resultados contendo o título, URL e citação.	Pass
Paginação de Resultados	Realizar uma pesquisa que retorne mais de 10 resultados e clicar em "Próximo".	A página deve atualizar mostrando os resultados 11-20, mantendo a consistência da pesquisa.	Pass
Pesquisa Sem Resultados	Pesquisar por uma sequência de caracteres aleatória (ex: "xyz123abc").	O sistema deve exibir uma mensagem amigável indicando que não foram encontrados resultados. dn	Pass
Links para uma Página	Aceder a /links-to, inserir um URL indexado e pesquisar.	Deve ser listada a lista de URLs que contêm hiperligações para o URL pesquisado.	Pass
Indexação via Hacker News (REST)	Na página de resultados, clicar no botão "Indexar Top Stories".	O sistema deve contactar a API do Hacker News, encontrar notícias relacionadas com a *query* e confirmar a indexação dos novos URLs.	Pass

Resumo IA (REST)	Realizar uma pesquisa com resultados. Verificar a caixa "Resumo da IA" no topo.	Deve aparecer um texto gerado pela IA resumindo o contexto da pesquisa com base nos resultados apresentados.	Pass
Chat com IA (Streaming)	Aceder a /ai-mode e enviar uma pergunta ("Olá").	A resposta deve ser gerada e apresentada token a token (efeito de escrita) sem bloquear a página.	Pass
Estatísticas em Tempo Real (WebSocket)	Abrir /stats. Com o sistema em repouso (sem pesquisas), verificar o tráfego de rede (Network Tab do browser). Realizar uma pesquisa noutra aba.	Enquanto o sistema está parado, **não** devem chegar novas mensagens WebSocket. Assim que a pesquisa ocorre, deve chegar **imediatamente** um pacote JSON com a atualização.	Pass
Robustez do Downloader	Tentar indexar um URL que devolve erro 403 (bloqueio).	O *Downloader* deve tentar variantes do *User-Agent* (conforme implementado em _try_get) e, se falhar, não deve crashar o sistema.	Pass
Persistência	Reiniciar o componente *Barrel*.	Os dados indexados anteriormente permanecem disponíveis.	Pass

Fallback IA	Simular falha na API da Cloud	O sistema comuta para o modelo local (Ollama) se disponível.	Pass

2. Arquitetura de Software: Integração FastAPI com Servidor RPC/RMI

A arquitetura do sistema Googol na Meta 2 expande o modelo distribuído original através da introdução de um servidor web desenvolvido em FastAPI. A escolha desta framework deveu-se à sua elevada performance e à compatibilidade nativa com o ecossistema Python utilizado no backend. Do ponto de vista arquitetural, o servidor web não processa a lógica de negócio do motor de busca; em vez disso, atua estritamente como um cliente gRPC e uma interface de apresentação para o utilizador. O servidor web comunica exclusivamente com o componente Gateway, mantendo o isolamento total face aos Barrels e Downloaders.

Padrão MVC

Para garantir a modularidade, escalabilidade e facilitar a integração de novos membros na equipa, a aplicação web adota o padrão MVC (Model-View-Controller). Esta estrutura permite desacoplar o tratamento de pedidos HTTP da complexidade da comunicação distribuída. A organização do código reflete esta separação da seguinte forma:

Controllers (web/controllers/): Esta camada atua como o ponto de entrada da aplicação, onde são definidas as rotas e os endpoints HTTP utilizando o APIRouter do FastAPI. A responsabilidade principal dos controladores é a orquestração do fluxo do pedido, começando pela validação rigorosa dos dados recebidos, sejam eles parâmetros de pesquisa (q, p), variáveis de caminho ou dados de formulário. O controlador não executa lógica de negócio; em vez disso, instância os serviços necessários através de um mecanismo de injeção de dependências (como get_search_service()), garantindo que cada pedido opera com uma conexão válida ao backend. Por fim, o controlador decide o formato da resposta adequada ao cliente: pode renderizar HTML através de TemplateResponse para navegadores (como no search_controller), devolver dados estruturados em JSONResponse para integrações via API (como no api_controller) ou até iniciar um StreamingResponse para enviar dados gerados em tempo real, como acontece nas funcionalidades de IA.

Services (web/services/): A camada de serviços representa a integração crítica do sistema, isolando completamente o servidor web dos detalhes do protocolo gRPC. Cada serviço, como o SearchService ou o IndexService, gere autonomamente o ciclo de vida da sua conexão, inicializando canais e stubs para comunicar com o Gateway definido nas variáveis de ambiente. Mais do que simples proxies para chamadas RPC, estes componentes implementam lógica híbrida que pode combinar múltiplas fontes de dados. Um exemplo claro é o IndexService, que coordena a consulta à API HTTP do Hacker News e, subsequentemente, itera sobre os resultados para invocar o método gRPC putNew para cada URL, abstraindo esta complexidade do controlador. Adicionalmente, é nesta camada que reside o tratamento defensivo de erros, capturando exceções de rede (grpc.RpcError) e convertendo-as em respostas seguras ou mensagens de erro controladas, impedindo que falhas no sistema distribuído afetem a estabilidade do servidor web.

Models (web/models/): Os modelos definem as estruturas de dados Python (dataclasses) utilizadas internamente pela aplicação web, funcionando estritamente como DTOs (Data Transfer Objects). O seu propósito fundamental é o desacoplamento arquitetural: impedem que as estruturas geradas automaticamente pelo Protobuf (index_pb2) permeiem a camada de visualização, garantindo que alterações no ficheiro .proto não quebrem o frontend. Estes modelos, como SearchResult ou SystemStats, normalizam e estruturam a informação de forma conveniente para a aplicação web, agrupando dados dispersos (como listas de resultados, totais e paginação) em objetos coesos. Além disso, incluem métodos auxiliares de serialização, como to_dict(), que facilitam a conversão direta para JSON, sendo essenciais para uniformizar as respostas da API REST.

Templates (web/templates/): Correspondendo à camada de visualização (View), os templates utilizam o motor Jinja2 para renderizar dinamicamente o HTML entregue ao cliente. Estes ficheiros contêm a lógica de apresentação necessária para transformar os dados dos modelos em interfaces ricas, utilizando estruturas de controlo para iterar sobre listas de resultados ou exibir condicionalmente elementos de paginação e mensagens de erro. Para além da renderização estática no servidor, os templates funcionam como o ponto de ancoragem para a interatividade do lado do cliente. É através deles que o JavaScript é carregado e inicializado para funcionalidades avançadas, como a conexão aos WebSockets na página de estatísticas, permitindo a manipulação do DOM em tempo real para animar gráficos e contadores sem a necessidade de recarregar a página.

Camada de Serviços: A Ponte entre HTTP e RPC

A integração técnica entre o mundo HTTP (FastAPI) e o mundo RPC (gRPC) ocorre estritamente na camada de Services. Esta camada abstrai a complexidade da rede, ocultando dos controladores a existência de canais e stubs. Foram criados serviços dedicados para cada domínio funcional:

SearchService: Este serviço atua como intermediário na transformação de dados. Quando invocado, é responsável pelo marshaling dos parâmetros de pesquisa para mensagens Protobuf (SearchWordRequest) antes de executar o método remoto no Gateway. Inversamente, após a execução, realiza o unmarshaling da resposta gRPC bruta, convertendo-a em objetos de domínio Python (SearchResult) que são consumidos pelos templates.

IndexService: Centraliza a lógica de indexação. Quando um utilizador submete um URL via formulário HTTP, este serviço valida o pedido e realiza o marshaling dos dados para invocar o método RPC putNew no Gateway, inserindo o URL na fila de processamento distribuída.

3. Integração de WebSockets

Para cumprir o requisito de monitorização em tempo real do sistema Googol, foi implementada uma arquitetura de comunicação assíncrona baseada em WebSockets. Ao contrário do modelo HTTP tradicional (pedido-resposta), esta abordagem estabelece um canal bidirecional e persistente (full-duplex) entre o navegador do cliente e o servidor Web, permitindo a atualização instantânea das métricas do sistema, como as pesquisas mais frequentes, os Barrels ativos e os tempos de resposta, sem a necessidade de recarregar a página.

A implementação no backend centra-se no StatsController (web/controllers/stats_controller.py), que define o endpoint /ws/stats. Tirando partido da natureza assíncrona do FastAPI, este controlador gere a conexão WebSocket através de um ciclo de vida não bloqueante. Uma vez estabelecida a ligação, o servidor entra num ciclo infinito (while True) que orquestra a obtenção de dados. É aqui que ocorre a integração crítica com o sistema distribuído: dentro deste ciclo, o controlador invoca o StatsService, que por sua vez executa uma chamada gRPC síncrona (getStatistics) ao componente Gateway. Este fluxo demonstra a capacidade do FastAPI em gerir conexões de longa duração (WebSockets) enquanto consome serviços RPC síncronos de forma eficiente.

Para otimizar a largura de banda e reduzir a carga no cliente, foi desenvolvida uma estratégia de "deteção de mudanças" no servidor. Em vez de enviar dados cegamente em cada iteração do ciclo, o controlador serializa o estado atual das estatísticas (obtido via RPC) numa representação comparável (JSON string). O sistema mantém em memória o último snapshot enviado e apenas transmite uma nova mensagem pelo WebSocket se houver uma divergência efetiva nos dados. Esta lógica transforma um processo de polling interno numa experiência event-driven para o frontend, garantindo que a rede apenas é utilizada quando o estado do sistema distribuído sofre alterações reais.

Do lado do cliente, a interface (web/templates/StatsPage/stats.html) utiliza JavaScript para gerir a conexão. O script estabelece o socket e define event listeners para processar as mensagens recebidas. Ao receber um novo pacote de dados JSON, o cliente não se limita a substituir o HTML; implementa lógica de manipulação do DOM para animar os contadores numéricos e as barras de progresso das pesquisas, proporcionando uma visualização fluida. O cliente inclui ainda mecanismos de robustez, como a tentativa automática de reconexão (exponential backoff) caso a ligação ao servidor Web seja interrompida, garantindo que o painel de administração recupera a sua funcionalidade sem intervenção do utilizador.

4. Integração de REST Web Services

Nesta secção, descreve-se como o sistema Googol foi integrado com serviços externos através de APIs REST, cumprindo o requisito de enriquecimento da plataforma com fontes de dados e inteligência externas. A implementação seguiu o padrão MVC, onde os *Services* (web/services/) executam as chamadas HTTP e processam a lógica, enquanto os *Controllers* (web/controllers/) expõem essas funcionalidades à interface.

4.1. Integração com Hacker News API

A integração com o Hacker News permite que os utilizadores indexem automaticamente as "top stories" atuais que correspondam a um termo de pesquisa, estando esta lógica implementada na classe IndexService localizada no ficheiro web/services/index_service.py.

O funcionamento deste mecanismo baseia-se no método index_hacker_news(query, limit), que realiza um pedido HTTP GET ao endpoint público da Algolia API (http://hn.algolia.com/api/v1/search), incluindo parâmetros como o termo de pesquisa (query), a tag story e o limite de resultados (hitsPerPage) para filtrar a resposta.

Após processar a resposta JSON e extrair os URLs das notícias encontradas, cada URL válido é imediatamente submetido para a fila de indexação do sistema Googol através da chamada gRPC putNew, reutilizando assim a infraestrutura de crawling já existente.

Por fim, o serviço assegura a gestão de erros utilizando a biblioteca requests do Python para tratar exceções de falhas de rede (requests.RequestException) ou códigos de estado HTTP inesperados, garantindo que eventuais falhas do serviço externo não comprometem a estabilidade do Googol.

4.2. Integração com IA Generativa (Ollama/Gemini/Groq)

Para fornecer resumos contextualizados e um assistente de chat, o sistema integra APIs de Inteligência Artificial Generativa, cuja lógica central se encontra implementada na classe AIService, localizada no ficheiro web/services/ai_service.py.

Uma característica distintiva desta arquitetura é o seu sistema robusto de fallback automático, onde o serviço tenta contactar os fornecedores numa ordem de prioridade definida: inicia com a Groq API (Modelo Llama-3) para garantir respostas rápidas de baixa latência, seguida pela

Google Gemini API como alternativa principal e, por fim, recorre ao Ollama (Local) caso não existam chaves de API configuradas ou ocorram falhas de internet.

Ao nível das funcionalidades, destaca-se o método generate_search_summary, que recebe a query do utilizador e os resultados da pesquisa para solicitar à IA um resumo conciso em português, bem como o assistente de chat, que utiliza Streaming Responses (Server-Sent Events) através do método stream_chat_with_history para que a resposta apareça progressivamente no browser, melhorando a experiência de utilização, sendo toda esta comunicação com o frontend efetuada através de endpoints REST.

5. Distribuição de tarefas pelos elementos do grupo

A divisão do trabalho para a Meta 2 foi planeada para potenciar a especialização de cada elemento, separando claramente a engenharia de backend da engenharia de frontend. O aluno **João Canais** assumiu a responsabilidade integral pela camada de visualização (View) e pela experiência do utilizador. O seu trabalho focou-se na construção da interface gráfica, desenvolvendo toda a estrutura HTML e a estilização CSS para garantir um design moderno e responsivo, adaptável a diferentes dispositivos. Foi também responsável pela criação e organização dos templates Jinja2, assegurando que a apresentação visual dos dados fosse coerente e intuitiva, traduzindo os requisitos funcionais numa experiência de navegação fluida e acessível para o utilizador final.

Por sua vez, o aluno **Francisco Gouveia** encarregou-se da arquitetura de software e da lógica de negócio da aplicação Web. Foi responsável pela implementação do padrão MVC sobre a framework FastAPI, estruturando os controladores (Controllers) e os modelos de dados (Models) que suportam o sistema. A sua contribuição abrangeu ainda a integração complexa de serviços, incluindo a comunicação gRPC com o backend existente, a implementação dos WebSockets no servidor para a gestão de estatísticas em tempo real e a integração com as APIs externas (Hacker News e IA). Desta forma, garantiu que toda a infraestrutura lógica e de comunicação estivesse robusta e devidamente desacoplada da interface.